3. What Is the Purpose of `async` and `await` Keywords in C#?

Introduction

In C#, `async` and `await` are keywords used to write asynchronous code. They allow an application to perform operations that may take time, such as database queries, file operations, or HTTP requests, without blocking the current thread while waiting for those operations to complete.

1. The Purpose of `async`

The `async` keyword is used to mark a method, lambda expression, or anonymous function as asynchronous. It allows the method to use the `await` keyword.

An asynchronous method commonly returns `Task`, `Task<T>`, or, in some cases, `ValueTask` or `ValueTask<T>`.

Example:

```
public async Task<string> GetMessageAsync()
{
    await Task.Delay(1000);
    return "Operation completed";
}
```

2. The Purpose of `await`

The `await` keyword asynchronously waits for an awaitable operation to complete. When the operation is incomplete, the method can suspend and return control to its caller instead of blocking the current thread.

After the operation completes, execution resumes from the point where it was suspended.

3. Why Do We Use `async` and `await`?

- **Improved responsiveness:** Applications can remain responsive while waiting for I/O operations.
- **Better scalability:** In web applications, threads do not need to remain blocked while waiting for supported asynchronous I/O operations, helping the server handle more concurrent requests.
- **Readable code:** Asynchronous operations can be written in a sequential style without manually managing callbacks.

- **Efficient I/O handling:** They are useful for database queries, HTTP requests, and file operations that involve waiting.

Important: `async` and `await` do not automatically make CPU-intensive work run on another thread. Their main benefit is allowing asynchronous operations to be awaited without blocking the current thread.

4. Example in ASP.NET Core MVC

Consider retrieving a student from a database using Entity Framework Core:

```
public async Task<IActionResult> ShowDetails(int id)
{
    var student = await _context.Students
        .Include(s => s.Department)
        .FirstOrDefaultAsync(s => s.Id == id);

    if (student == null)
        return NotFound();

    return View(student);
}
```

In this example:

- `async` allows the action method to use `await`.
- `FirstOrDefaultAsync()` executes the database query asynchronously.
- `await` waits for the query to finish without blocking the current thread while the database operation is pending.
- `NotFound()` returns an HTTP 404 response if the student does not exist.
- `View(student)` displays the retrieved student's information.

5. Synchronous vs. Asynchronous Operations

Synchronous	Asynchronous
A thread may remain blocked while waiting for an operation.	The method can suspend while an asynchronous operation is pending.
Can limit scalability when many requests wait for I/O.	Can improve scalability for I/O-bound workloads.
Often uses methods such as `ToList()`.	Often uses methods such as `ToListAsync()`.

Conclusion

The purpose of `async` and `await` in C# is to make asynchronous programming easier to write and understand. In ASP.NET Core applications, they are particularly useful for

database and other I/O operations because they help maintain responsiveness and improve server scalability without unnecessarily blocking threads.